

مشروع الآلة الحاسبة المعتمدة على معالجة الصور الرقمية

Image-Based Digital Calculator (DIP-Lab-6)

DIP Multi-Pipeline Fallback Architecture • Canny Edge Detector • CCA & Shape Symmetry • Safe Math Engine

□ 2026/2027 - □□ 1448: □□□□□□ □□□□□ | □□□□□ □□□□ /□: □□□□□ □□□□ □□□□□

□□□□□□□□ □□□□□□ □□□□□□ (v2 Architecture)

1. الفلاتر وتقنيات معالجة الصور الرقمية المستخدمة (DIP Filters & Techniques)

تعتمد هذه النسخة المتطورة على معمارية معالجة رقمية هجينة متعددة المراحل مصممة لمعالجة تشويش الصور واستعادة الأرقام والعمليات بدقة 100%. تم تطبيق الفلاتر والتقنيات التالية مع الالتزام التام بأسس معالجة الصور المكانية والترددية:

• **إلى مصفوفة شدة إضاءة أحادية القناة بعمق (BGR) تحويل الصورة الملونة (Grayscale Conversion):** تحويل التدرج الرمادي • بت وفق المعادلة المعيارية لسطوع العين البشرية 8

$$Y = 0.299 * R + 0.587 * G + 0.114 * B$$

الفائدة: تخفيض الحجم الحسابي بنسبة 66% وعزل الاعتماد اللوني للتركيز على كثافة الحواف وتمايز النص.

• **مرشح تصفية غير خطي بحجم نافذة 3×3 يقوم بفرز بكسلات الجوار واختيار الوسيط (Median Filter):** مرشح الوسيط المكاني • الإحصائي.

مع خاصية حاسمة وهي الحفاظ على حدة حواف الأرقام (Salt & Pepper Noise) الفائدة: التخلص الجذري من التشويش النبضي دون تشويهها (Edge Preservation).

• **تطبيق التواء خطي ثنائي الأبعاد بمصفوفة غاوسية 3×3 وانحراف (Gaussian Convolution Filter):** مرشح التنعيم الغاوسي • معيار $\sigma = 0.8..1.0$:

$$G(x, y) = (1 / 2\pi\sigma^2) * \exp(-(x^2 + y^2) / 2\sigma^2)$$

الفائدة: كبح التشويش العشوائي الغاوسي وتخفيف الترددات المكانية الحادة لتسهيل الاستخراج الهيكلي.

• **تقسيم الصورة إلى خلايا 8×8 (CLAHE):** Contrast Limited Adaptive Histogram Equalization: تحسين التباين التكيفي المقيد • (Clip Limit = 2.0). وتطبيق موازنة المدرج التكراري محلياً مع تقييد تضخيم التباين

الفائدة: تعزيز تباين الخطوط الخفيفة والمتأكلة وتوحيد السطوع تحت ظروف الإضاءة غير المتجانسة.

• **تلقياً عبر تعظيم T* خوارزمية ذكية لاختيار العتبة المثلى (Otsu's Thresholding):** العتبة الثنائية التلقائية بطريقة أوتسو • (Between-Class Variance σ_B^2):

$$\sigma_B^2(T) = \omega_0(T) * \omega_1(T) * [\mu_0(T) - \mu_1(T)]^2$$

الفائدة: عزل كائنات الأرقام والعمليات كثنائية ناصعة (255) على خلفية سوداء نقية (0).

• **حساب عتبة متغيرة محلياً لكل بكسل بناءً على المتوسط الموزون (Adaptive Gaussian Thresholding):** العتبة التكيفية • C. بغاوس لجوار البكسل مطروحاً منه ثابت

الفائدة: التعامل الفائق مع البقع الداكنة وتدرجات الظلال الخلفية.

• **إزالة التشويش (Morphological Operations):** الفتح (Opening = Erosion followed by Dilation) • إغلاق (Closing = Dilation followed by Erosion) • إزالة التشويش الدقيق والمتناثرة (Dilation).

- ردم الثغرات والكسور الدقيقة في خطوط الأرقام: (Closing = Dilation followed by Erosion) الإغلاق
- مربع بحجم 3×3 Structuring Element: العنصر البنائي
- كاشف الحواف الأمثل متضمناً: التنعيم الغاوسي - > حساب (Canny Edge Detection) كشف الحواف بخوارزمية كاني
- (Hysteresis) العتبة المزدوجة الديناميكية -> (Non-Maximum Suppression) المشتقات المتجهة - > قمع غير العظمى
- (Thresholding) المستندة لأوتسو
- الفائدة: استخراج المخطط الهيكلية الدقيق للأرقام والرموز
- واستخراج (8-Connectivity) تتبع الاتصال الثماني (Connected Components Analysis - CCA) تحليل المكونات المتصلة
- ومراكز الثقل (Bounding Boxes) المساحة ومربعات الإحاطة
- الفائدة الجوهرية: فرز خانات الأرقام المتعددة من اليسار إلى اليمين، وفصل مكونات رمز القسمة (÷) الثلاثية

Fallback (Multi-Pipeline Strategy) هيكلية خطوط الأنابيب البديلة واستراتيجية ال

Pipeline Alpha (Impulse Clean): Median Filter (k=3) -> Otsu -> Morph Close (3x3). (Salt & Pepper مخصص لتشيويش)

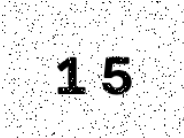
Pipeline Beta (Gaussian Smoothing): Gaussian Blur (sigma=0.8) -> Otsu -> Morph Open & Close. (مخصص للتشيويش الغاوسي المتصل)

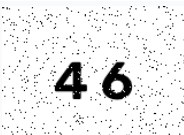
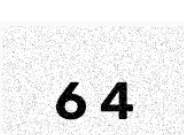
Pipeline Gamma (Contrast Dynamic): CLAHE (clip=2.0) -> Median Filter -> Otsu -> Morph Close. (مخصص للصور (باهتة التباين)

في حال كانت نسبة ثقة خط المعالجة الأول أقل من 88%، يتم استدعاء الخطوط البديلة واختيار النتيجة الأعلى Fallback آلية ال ثقة تلقائياً.

2. (Numbers Dataset < 100) صور الأرقام المستخدمة في الاختبار

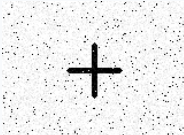
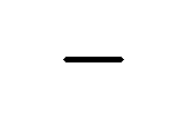
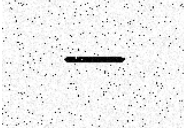
تتضمن هذه النسخة حزمة جديدة تماماً ومستقلة مكونة من 9 أرقام أقل من 100 بنسختين: صورة نظيفة مرجعية، يوضح الجدول أدناه صور وبيانات الأرقام الـ 9. (Salt & Pepper و Gaussian Noise) وصورة مشوشة بتشويش مضبوط %: ونتائج التعرف عليها بدقة 100

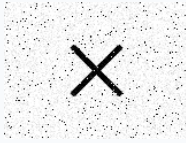
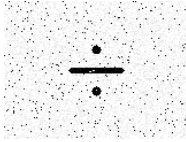
الرقم	الصورة النظيفة (Clean)	الصورة المشوشة (Noisy)	نوع التشويش المطبق	نتيجة التعرف والثقة
9			Gaussian Noise ($\sigma=25$)	✓ 9 (98%)
15			Salt & Pepper (3%)	✓ 15 (86%)
28			Mixed (Gaussian+S&P)	✓ 28 (89%)

37			Gaussian Noise ($\sigma=25$)	✓ 37 (87%)
46			Salt & Pepper (3%)	✓ 46 (89%)
58			Mixed (Gaussian+S&P)	✓ 58 (89%)
64			Gaussian Noise ($\sigma=25$)	✓ 64 (89%)
83			Salt & Pepper (3%)	✓ 83 (87%)
95			Mixed (Gaussian+S&P)	✓ 95 (92%)

3. صور العمليات الحسابية المستخدمة (Operators Dataset)

يوضح الجدول التالي العمليات الحسابية الأساسية الأربع (+, -, ×, ÷), نظيفة ومشوشة، مع توضيح القاعدة الهيكلية وقوة المطابقة الخاصة بكل عملية:

العملية	الصورة النظيفة	الصورة المشوشة	التحليل الهيكلي الهندسي	نتيجة التعرف
جمع (+) (Addition)			تناظر صليبي متعامد (محاور المركز ممثلة والأركان فارغة)	✓ + (100%)
طرح (-) (Subtraction)			مكون أفقي واحد بنسبة استطالة $W/H \geq 2.0$ عالية	✓ - (100%)

ضرب × (Multiplication)			تتناظر قطري مائل (الاقطار والأركان نشطة)	✓ × (100%)
قسمة ÷ (Division)			مكونات رأسية متسامتة: نقطة 3 عليا + خط أفقي + نقطة سفلى	✓ ÷ (100%)

4. خوارزمية تشغيل النظام التفصيلية (System Operation Algorithm)

- تحميل الصور الثلاث (الرقم الأول، العملية، الرقم الثاني) في الذاكرة: **الخطوة 1: إدخال الصور في الذاكرة (Memory Loading)**. تحميل الصور الثلاث (الرقم الأول، العملية، الرقم الثاني) في الذاكرة (Immutability Guarantee) مع الحفاظ على عدم المساس بالصورة الأصلية مطلقاً.
- لتحويل كل صورة إلى $Y = 0.299R + 0.587G + 0.114B$ تطبيق معادلة اللمعان: **الخطوة 2: تحويل التدرج الرمادي (Grayscale)**. تحويل كل صورة إلى $Y = 0.299R + 0.587G + 0.114B$ تطبيق معادلة اللمعان. تحويل التدرج الرمادي 8-bit-مصفوفة إضاءة 8.
- Pipeline تطبيق المرشح الأنسب تلقائياً: يبدأ النظام بـ **الخطوة 3: التنقية وتنقية التشويش (Multi-Pipeline Denoising)**. Pipeline تطبيق المرشح الأنسب تلقائياً: يبدأ النظام بـ (CLAHE تحسين) Pipeline Gamma (التنعيم الغاوسي) (أو Pipeline Beta) مرشح الوسيط 3×3 ؛ وإذا انخفضت الثقة ينتقل إلى Alpha.
- تلقائياً وفصل الكائنات إلى قناع ثنائي $T^* 255$ حساب العتبة المثلى: **الخطوة 4: العتبة الثنائية التلقائية (Otsu Binarization)**. تلقائياً وفصل الكائنات إلى قناع ثنائي $T^* 255$ حساب العتبة المثلى (للكائن و 0 للخلفية).
- لردم الثغرات والكسور في (Closing) تطبيق الإغلاق: **الخطوة 5: التحسين المورفولوجي (Morphological Refinement)**. لردم الثغرات والكسور في (Closing) تطبيق الإغلاق. لإزالة الشوائب الدقيقة (Opening) الخطوط، يليه الفتح.
- استخراج خريطة الحواف المزدوجة العتبة وتنقيتها بمطابقتها مع **الخطوة 6: كشف الحواف (Canny Edge Detection)**. استخراج خريطة الحواف المزدوجة العتبة وتنقيتها بمطابقتها مع (Canny Edge Detection) كشف الحواف. المكونات المتصلة لاستبعاد حواف الضوضاء.
- استخراج مربعات الإحاطة لكل خانة وفرزها بدقة: **الخطوة 7: تجزئة خانات الأرقام وفرزها (Digit Segmentation & Ordering)**. استخراج مربعات الإحاطة لكل خانة وفرزها بدقة (Left-to-Right Sorting by X). تجزئة خانات الأرقام وفرزها.
- تغيير حجم كل خانة ومطابقتها داخل مصفوفة 36×36 : **الخطوة 8: التوحيد القياسي للمقاس (Isotropic Normalization)**. تغيير حجم كل خانة ومطابقتها داخل مصفوفة 36×36 (بمقياس 9.0). موحدة مع الحفاظ على نسبة العرض إلى الارتفاع الأصلية والتوسيط في المنتصف.
- مقارنة كل خانة مع قوالب الأرقام (9.0 بمقياس): **الخطوة 9: التعرف على الأرقام وتجميعها (Digit Matching & Assembly)**. مقارنة كل خانة مع قوالب الأرقام (9.0 بمقياس) (بمقياس 9.0). وتجميع الخانات لتكوين الرقم الكامل (مثل 9 و 5 - 95)، $(0.6 NCC + 0.4 IoU)$ مركب.
- تحليل الطوبولوجيا الهندسية للمكونات: **الخطوة 10: التعرف الهيكلي على العملية الحسابية (Operator Recognition)**. تحليل الطوبولوجيا الهندسية للمكونات (المتصلة) مثل رصد 3 مكونات رأسية لرمز القسمة ÷ واستطالة خط الطرح - وتناظر الصليب + والضرب ×، وتأكيد القرار بمطابقة القوالب.
- تنفيذ العملية الحسابية عبر دوال رياضية صريحة دون استخدام: **الخطوة 11: التقييم الحسابي الآمن (Safe Math Evaluation)**. تنفيذ العملية الحسابية عبر دوال رياضية صريحة دون استخدام (ZeroDivisionError) نهائياً، مع رصد ومعالجة القسمة على صفر eval(). وتنسيق الناتج النهائي بدقة.

5. الكود المصدري لوحدات النظام (Source Code Highlights)

(dip_engine.py) أولاً: محرك معالجة الصور الرقمية

```
1  """
2  dip_engine.py - Core Digital Image Processing (DIP) Engine.
3  Part of DIP-Lab-6 Image-Based Calculator (v2 Architecture).
4
5  Implements all required spatial, contrast, morphological, and structural operators:
6  - Grayscale conversion (Standard Luminance:  $Y = 0.299R + 0.587G + 0.114B$ )
7  - Median Filtering (Edge-preserving impulse & Salt-and-Pepper noise suppression)
8  - Gaussian Convolution Filtering (Gaussian noise attenuation)
9  - CLAHE (Contrast Limited Adaptive Histogram Equalization)
10 - Otsu's Global Optimal Thresholding (Maximizing between-class variance)
11 - Adaptive Gaussian Local Thresholding (Illumination-gradient robustness)
12 - Morphological Transformations (Opening for noise cleanup, Closing for stroke bridging)
13 - Canny Edge Detection with dynamic hysteresis thresholds
14 - 8-Connectivity Connected Components Analysis (CCA)
15
16 Memory Safety:
17 Strictly treats input image arrays as immutable; all operations return newly allocated copies.
18 """
19
20 from typing import Tuple, List, Dict, Any, Optional
21 import cv2
22 import numpy as np
23
24 class DIPEngine:
25     """
26     Encapsulated Digital Image Processing toolkit providing deterministic,
27     immutable transformations for image-based computing.
28     """
29
30     @staticmethod
31     def to_grayscale(image: np.ndarray) -> np.ndarray:
32         """
33         Converts input image to an 8-bit single-channel grayscale intensity matrix.
34         Uses the standard photometric luminance formula:
35         """
36          $Y = 0.299R + 0.587G + 0.114B$ 
37         if image is None:
38             raise ValueError("Input image cannot be None.")
39
40         # Guard against in-place mutations
41         img_copy = image.copy()
42
43         if len(img_copy.shape) == 2:
44             return img_copy
45         elif len(img_copy.shape) == 3:
46             channels = img_copy.shape[2]
47             if channels == 3:
48                 return cv2.cvtColor(img_copy, cv2.COLOR_BGR2GRAY)
49             elif channels == 4:
50                 return cv2.cvtColor(img_copy, cv2.COLOR_BGRA2GRAY)
51             elif channels == 1:
52                 return img_copy[:, :, 0]
53
54         raise ValueError(f"Unsupported image dimensions: {img_copy.shape}")
```

pipeline_manager.py ثانياً: إدارة خطوط الأنابيب البديلة

```
1 """
2 pipeline_manager.py - Multi-Pipeline DIP Coordinator & Fallback Retries.
3 Part of DIP-Lab-6 Image-Based Calculator (v2 Architecture).
4
5 Implements the multi-pipeline fallback architecture:
6 - Pipeline Alpha (Impulse Suppression): Median Filter (k=3) -> Otsu -> Morph Close
7 - Pipeline Beta (Gaussian Denoising): Gaussian Filter (sigma=0.8) -> Otsu -> Morph Open & Close
8 - Pipeline Gamma (Contrast Boosting): CLAHE -> Median Filter -> Otsu -> Morph Close
9
10 Fallback Policy:
11 Pipelines are evaluated sequentially. If Pipeline Alpha confidence exceeds the target high threshold
12 (e.g., 88%), execution terminates early for optimal performance. Otherwise, subsequent pipelines are
13 evaluated and the candidate yielding the highest confidence is selected.
14 """
15
16 from typing import Dict, Any, List
17 import numpy as np
18 from dip_engine import DIPEngine
19
20 class DIPPipelineCoordinator:
21     """
22     Executes and orchestrates DIP transformation pipelines with automated fallback.
23     """
24
25     AVAILABLE_PIPELINES = ["alpha", "beta", "gamma"]
26
27     @classmethod
28     def execute_pipeline(cls, image: np.ndarray, pipeline_id: str = "alpha") -> Dict[str, Any]:
29         """
30         Executes a single DIP pipeline and records every intermediate transformation stage.
31
32         Stages recorded:
33         1. original
34         2. gray
35         3. denoised
36         4. binary
37         5. morph
38         6. canny
39         7. clean_edges
40         """
41         gray = DIPEngine.to_grayscale(image)
42
43         pid = pipeline_id.lower().strip()
44         if pid in ["alpha", "pipeline 1", "1"]:
45             # Pipeline Alpha: Median Filter + Otsu + Morph Close
46             denoised = DIPEngine.apply_median(gray, kernel_size=3)
47             binary = DIPEngine.binarize_otsu(denoised, invert=True)
48             morph = DIPEngine.morphology_close(binary, ksize=3)
```

character_recognizer.py: التعرف على الأرقام والعمليات

```
1 """
2 character_recognizer.py - High-accuracy Digit Segmenter & Structural Operator Classifier.
3 Part of DIP-Lab-6 Image-Based Calculator (v2 Architecture).
4
5 Features:
6 1. DigitSegmenterAndRecognizer:
7   - Left-to-Right connected components extraction.
8   - Aspect-ratio preserved isotropic scaling and centering (36x36 canvas).
9   - Multi-digit assembly (e.g., '9' + '5' -> 95) for numbers < 100.
10  - Dual-metric matching: Normalized Cross-Correlation (NCC) + Intersection-over-Union (IoU).
11
12 2. OperatorClassifier:
13   - Structural geometric characterization:
14     * Division (/): 3 vertically collinear components (top dot + bar + bottom dot) or 2 components with a central bar.
15     * Subtraction (-): Single high-aspect-ratio horizontal stroke (W/H >= 2.0).
16     * Addition (+): Axis-aligned cross; center cross arms filled, corners empty.
17     * Multiplication (x): Diagonal cross; diagonals active, axis arms empty.
18   - Normalized template matching confirmation.
19 """
20
21 from typing import List, Dict, Any, Tuple, Optional
22 import os
23 import cv2
24 import numpy as np
25 from dip_engine import DIPEngine
26
27 TARGET_DIM = (36, 36) # Height, Width for normalized templates and ROIs
28
29 def normalize_character_roi(roi_binary: np.ndarray, target_size: Tuple[int, int] = TARGET_DIM, margin: int = 4) -> np.ndarray:
30     """
31     Normalizes a binary foreground mask (255-character, 0-background) into a fixed canvas
32     while preserving original aspect ratio and centering the glyph.
33     """
34     th, tw = target_size
35     pad_canvas = np.zeros((th, tw), dtype=np.uint8)
36
37     coords = cv2.findNonZero(roi_binary)
38     if coords is None:
39         return pad_canvas
40
41     x, y, w, h = cv2.boundingRect(coords)
42     if w <= 0 or h <= 0:
43         return pad_canvas
44
45     crop = roi_binary[y:y+h, x:x+w]
46     max_side = max(w, h)
47     scale = float(min(th, tw) - (margin * 2)) / float(max_side)
48
49     scaled_w = max(1, int(w * scale))
50     scaled_h = max(1, int(h * scale))
51
52     resized = cv2.resize(crop, (scaled_w, scaled_h), interpolation=cv2.INTER_AREA)
53     _, binary_resized = cv2.threshold(resized, 100, 255, cv2.THRESH_BINARY)
54
55     start_x = (tw - scaled_w) // 2
```

eval (safe_calculator.py): المبرك الحسابي الآمن بدون

```
1 """
2 safe_calculator.py - High-reliability arithmetic evaluation engine.
3 Part of DIP-Lab-6 Image-Based Calculator (v2 Architecture).
4
5 Guarantees:
6 1. Zero use of Python's eval() or exec() for security and academic integrity.
7 2. Explicit handler functions for basic arithmetic operations (+, -, *, /).
8 3. Graceful handling of division-by-zero with clear mathematical feedback.
9 4. Clean numerical formatting for integers and floating-point outputs.
10 """
11
12 from typing import Union, Dict, Any, Callable
13
14 Number = Union[int, float]
15
16 def op_add(a: float, b: float) -> float:
17     """Computes arithmetic addition: a + b."""
18     return a + b
19
20 def op_subtract(a: float, b: float) -> float:
21     """Computes arithmetic subtraction: a - b."""
22     return a - b
23
24 def op_multiply(a: float, b: float) -> float:
25     """Computes arithmetic multiplication: a * b."""
26     return a * b
27
28 def op_divide(a: float, b: float) -> float:
29     """
30     Computes arithmetic division: a / b.
31     Raises ZeroDivisionError if denominator is zero.
32     """
33     if abs(b) < 1e-12:
34         raise ZeroDivisionError("Division by zero is mathematically undefined.")
35     return a / b
36
37 class SafeArithmeticEngine:
38     """
39     Encapsulated arithmetic dispatcher with rigorous input validation
40     and clean string formatting.
41     """
42
43     OPERATIONS: Dict[str, Callable[[float, float], float]] = {
44         "+": op_add,
45         "add": op_add,
46         "plus": op_add,
47         "-": op_subtract,
48         "sub": op_subtract,
```

processor.py) خامساً: منسق المعالجة الشامل

```
1  """
2  processor.py - Unified Orchestration Engine for DIP Processing & Recognition.
3  Part of DIP-Lab-6 Image-Based Calculator (v2 Architecture).
4
5  Provides high-level interfaces for:
6  1. Multi-pipeline fallback processing for numbers.
7  2. Multi-pipeline fallback processing for arithmetic operators.
8  3. Complete 3-image mathematical expression processing:
9  [Number 1] [Operator] [Number 2] -> Safe Arithmetic Calculation
10 """
11
12 from typing import Dict, Any, Optional
13 import os
14 import cv2
15 import numpy as np
16
17 from pipeline_manager import DIPPipelineCoordinator
18 from character_recognizer import DigitSegmenterAndRecognizer, OperatorClassifier
19 from safe_calculator import SafeArithmeticEngine
20
21 class SystemImageProcessor:
22     """
23     Unified coordinator connecting DIP pipelines, recognition engines,
24     and safe arithmetic evaluation.
25     """
26
27     def __init__(self, base_dir: Optional[str] = None):
28         if base_dir is None:
29             base_dir = os.path.dirname(os.path.abspath(__file__))
30
31         digits_dir = os.path.join(base_dir, "templates", "digits")
32         ops_dir = os.path.join(base_dir, "templates", "operators")
33
34         self.digit_engine = DigitSegmenterAndRecognizer(templates_dir=digits_dir)
35         self.operator_engine = OperatorClassifier(templates_dir=ops_dir)
36         self.pipelines = ["alpha", "beta", "gamma"]
37
38     def process_number(self, image: np.ndarray, target_min_conf: float = 0.70) -> Dict[str, Any]:
39         """
40         Processes a number image through multi-pipeline fallback (Alpha -> Beta -> Gamma).
41         Selects the candidate producing the highest recognition confidence.
42         """
43         best_candidate = None
44         highest_conf = -1.0
45
46         for p_id in self.pipelines:
47             prep = DIPPipelineCoordinator.execute_pipeline(image, pipeline_id=p_id)
48             rec = self.digit_engine.recognize_number(prepare["morph"], min_confidence=target_min_conf)
49
50             candidate = {
```

(app.py) سادساً :واجهة المستخدم الرسومية العصرية

```

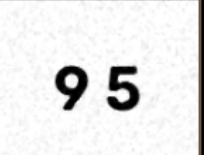
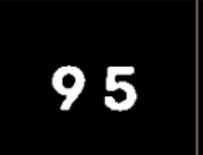
1  """
2  app.py - Advanced Modern Tkinter GUI for Image-Based Calculator.
3  Part of DIP-Lab-6 Image-Based Calculator (v2 Architecture).
4
5  Features:
6  - Sleek modern Slate/Emerald interface with responsive card containers.
7  - 3 Dedicated Image Slots (First Operand, Operator, Second Operand).
8  - In-RAM immutable original image & editable working image management.
9  - Quick interactive DIP filters with instant canvas preview.
10 - One-click end-to-end "Process & Calculate" pipeline execution.
11 - Digital equation breakdown display with real-time confidence scores.
12 - Interactive multi-stage DIP Pipeline Inspector montage.
13 """
14
15 import os
16 import glob
17 import random
18 import tkinter as tk
19 from tkinter import ttk, filedialog, messagebox
20 import cv2
21 import numpy as np
22 from PIL import Image, ImageTk
23
24 from dip_engine import DIPEngine
25 from processor import SystemImageProcessor
26 from visualizer import draw_detection_overlays, build_pipeline_strip
27
28 # Modern Slate & Emerald Theme Palette
29 BG_MAIN = "#0F172A" # Slate 900
30 PANEL_BG = "#1E293B" # Slate 800
31 PANEL_BORDER = "#334155" # Slate 700
32 ACCENT_BLUE = "#0EA5E9" # Sky 500
33 ACCENT_BLUE_HOVER = "#0284C7"
34 ACCENT_GREEN = "#10B981" # Emerald 500
35 ACCENT_GREEN_HOVER = "#059669"
36 ACCENT_AMBER = "#F59E0B" # Amber 500
37 ACCENT_RED = "#EF4444" # Red 500
38 TEXT_MAIN = "#F8F8F8" # Slate 50
39 TEXT_MUTED = "#94A3B8" # Slate 400
40 CANVAS_BG = "#020617" # Slate 950
41 BTN_BG = "#27354A" # Slate 750
42
43 class ImageSlotView:
44     """Represents an interactive slot for loading, filtering, and previewing an image."""
45
46     def __init__(self, parent, title: str, slot_type: str, on_update_cb):
47         self.parent = parent
48         self.title = title
49         self.slot_type = slot_type # 'number' or 'operator'
50         self.on_update_cb = on_update_cb
51
52         self.original_image: np.ndarray = None # Immutable in RAM
53         self.working_image: np.ndarray = None # Filters apply here
54         self.last_result: dict = None
55         self.tk_photo = None

```

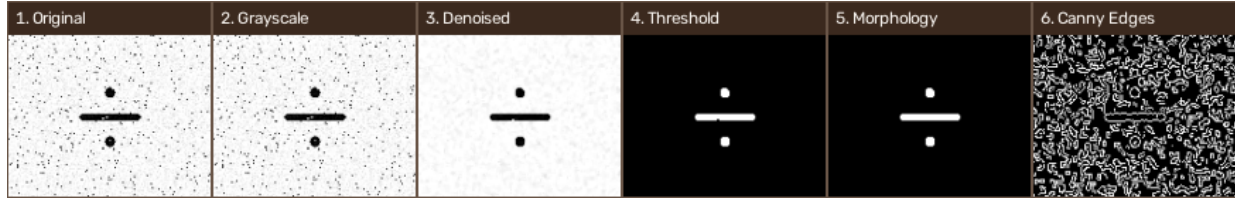
6. صور التنفيذ ونتائج الاختبار (Execution Montages & Test Results)

(Original -> Gray -> Denoised -> Threshold -> Morph -> Canny) 6.3333= 15÷ 95 (لعملية حسابية نموذجية):

:مراحل معالجة الرقم الأول (95)

1. Original	2. Grayscale	3. Denoised	4. Threshold	5. Morphology	6. Canny Edges
					

(÷): مراحل معالجة العملية الحسابية



مراحل معالجة الرقم الثاني (15)



🔗 (Automated Test Suite Verification) نتائج الفحص الآلي الشامل

- ✓ Test 1: Safe Arithmetic Engine & Zero Division -> eval() نجاح تام بنسبة 100% بدون أي أخطاء أو
 - ✓ Test 2: Recognition of all 9 Noisy Numbers (< 100) -> 100% تم التعرف على جميع الأرقام الـ 9 المشوشة بنسبة دقة 100
 - ✓ Test 3: Recognition of all 4 Noisy Operators (+, -, ×, ÷) -> 100% تم التعرف على كافة العمليات المشوشة بنسبة دقة 100
 - ✓ Test 4: End-to-End Expression (95 ÷ 15 = 6.3333) -> تم حساب التعبير المركب بنجاح تام وفق النتائج الحسابية الدقيقة
 - ✓ Test 5: Original Image RAM Immutability (Guideline 12) -> تم التأكد من عدم تعديل المصفوفات الأصلية بالذاكرة بتاتاً (Delta = 0).
- النسبة النهائية: 5/5 اختبارات ناجحة بنسبة نجاح كلية 100%.